

Com base na descrição do “Seu Cantinho” e os problemas que o sistema implementado emergencialmente enfrentaram, começamos listando os problemas e demandas que o novo sistema deveria ter, chegando no seguinte:

- resolver problema de double-booking
- evitar perda de informações financeira
- diminuir tempo de operação
- implementar uma interface simples de usar e gerenciar

além disso, por ser um sistema de reservas que envolve pagamentos e dados sigiloso de usuários, alguns outros requisitos estavam implícitos:

- Proteger dados de pagamentos e dados de clientes
- Suportar maior tráfego em alta temporada

Com isso, listamos os problemas a serem resolvido em um ASR's, definindo prioridades para cada um:

ASR's coletados

Alta prioridade

Confiabilidade: Evitar double-booking

Escalabilidade: Suportar operação centralidaza (3 estados)

Usabilidade: Ser simples de usar e gerenciar

Segunrança: Proteger dados de pagamentos e dados de clientes

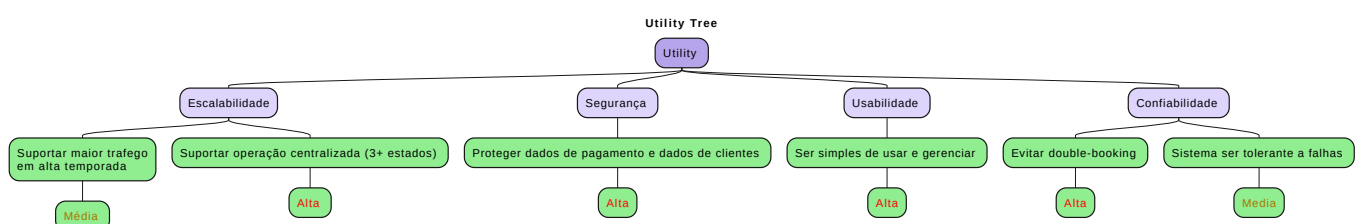
Média prioridade

Escalabilidade: Suportar maior tráfego em alta temporada

Confiabilidade: Sistema ser tolerante a falhas

E assim, definimos uma utility tree com os ASR's mais importantes:

Utility tree



Estilo arquitetural escolhido e trade offs

Com os ASR's definidos, partimos para discussão de qual estilo arquitetônico se encaixaria melhor no nosso sistema; Os principais focos da discussão foram o problema da double booking e da consistência de dados entre as filiais, o que nos levou a querer um sistema em que o banco de dados seja centralizado (assim ficaria mais fácil para implementar uma solução que evite double booking). Além disso, outros dois pontos que influenciaram nossa escolha foram a necessidade de simplicidade e o fato da escalabilidade ter ficado como ASR de prioridade média (escalabilidade elástica em alta temporada) o que nos levou a solução: **monolito utilizando MVC**

Com ele seria possível:

- Ter um banco de dados centralizado
 - Mantendo a consistência entre as filiais e evitando double booking
- Simplicidade
 - Monólito é simples de implementar o que possibilita a criação de um sistema simples de gerenciar e acessar de forma mais fácil
- Escalabilidade em alta temporada
 - Mesmo sendo o ponto fraco da arquitetura, é possível ter uma escalabilidade elástica do sistema em alta temporada (escalabilidade vertical)

Trade Offs

Apesar da arquitetura escolhida resolver os principais problemas do sistema, existem alguns trade-offs que devem ser considerados:

Simplicidade x Escalabilidade

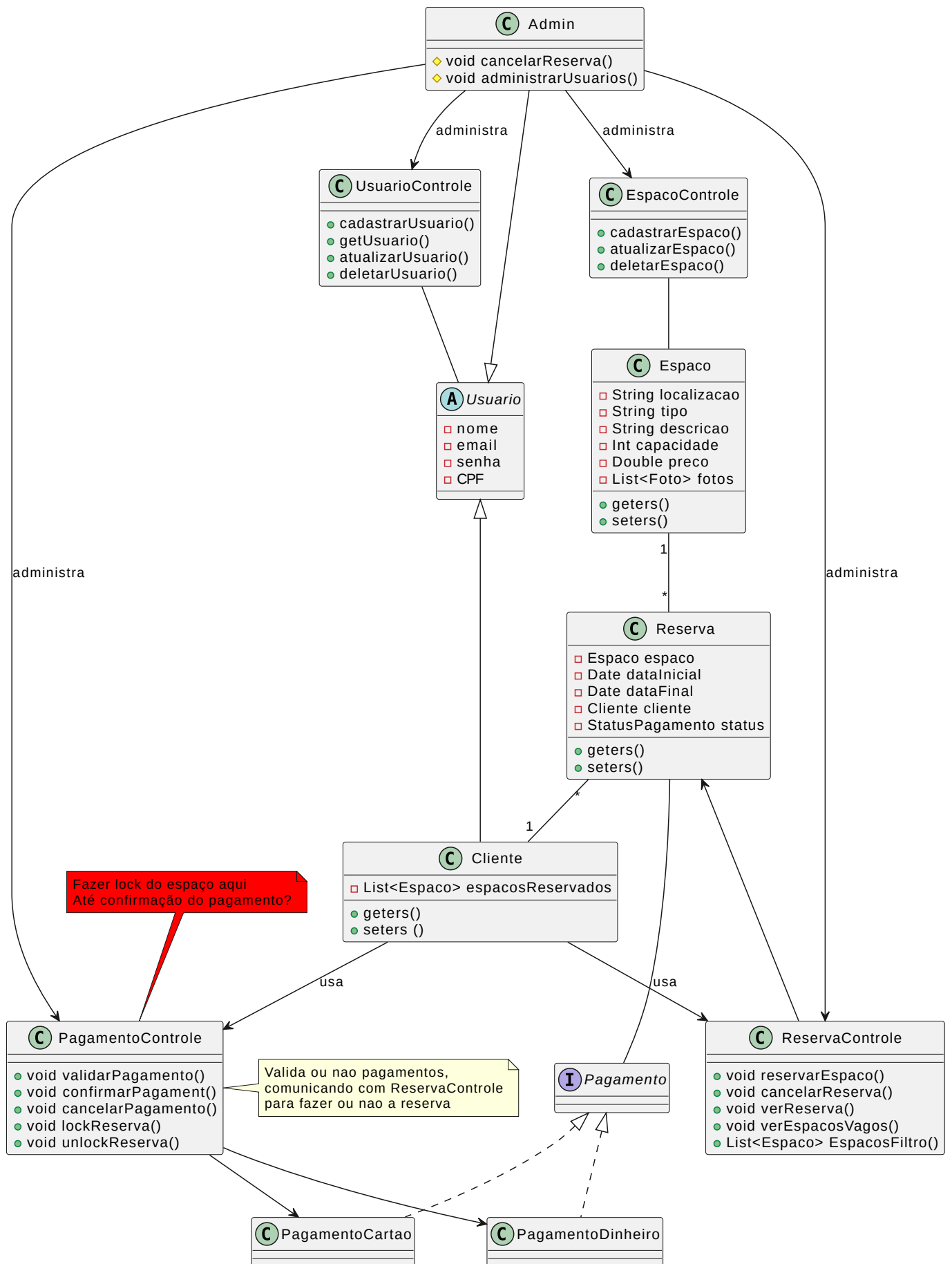
Apesar de ser uma arquitetura simples de implementar, a granularidade da escalabilidade é o maior problema. Se apenas um dos módulos estiverem recebendo muitas requisições e for preciso escalar o sistema, todo o sistema escala junto

Consistência x Desempenho

Ter um banco de dados central trás consistência para o sistema porém em troca de desempenho, com muitas requisições o sistema pode sofrer lentidão.

Diagrama de classes

Com isso feito, começamos a esboçar o diagrama de classes do sistema:



Alunos

(coloquei no fim porq tava quebrando a formatação lá em cima)

Patrick Oliveira Lemes GRR20211777

Nicolas Andre Rizzardi GRR20205509